

TECHNICAL CASE STUDY

Hartwell Mechanical Services Quote System

A full-stack system integrating HubSpot CMS, a custom Express API, and CRM automation for self-service quote acceptance with real-time pricing and invoice generation

--

Layer	Technology	Detail
Frontend	HubSpot CMS	HubL templates, Jinja2-style, modular components
Client Logic	Vanilla JS (ES6+)	<code>QuoteAcceptanceManager</code> class, no framework deps
Backend	Node.js / Express 4	RESTful API, CORS, rate limiting, in-memory cache
CRM	HubSpot CRM v3/v4	Standard + 2 custom objects, v3/v4 association APIs
Invoicing	HubSpot Invoices API	Auto line items, associations, status transitions
Cache	NodeCache	TTL-based: 3600s prod, 10s dev
Hosting	Render	Managed Node.js service

System Overview

The platform is a three-tier architecture. The presentation layer runs inside HubSpot CMS as a custom quote theme with six modular HubL components. Client-side JavaScript handles interactivity, real-time calculations, and state management. The application layer is a standalone Express API that acts as a secure intermediary between the browser and HubSpot APIs. The data layer uses HubSpot CRM for persistence, leveraging both standard objects (deals, contacts, companies, quotes, line items) and two custom objects (Accepted Quotes, Selected Products).

Frontend Architecture

Template Modules

The HubSpot quote template is composed of six independent modules, each with its own `fields.json` (schema), `module.html` (HubL markup), `module.css` (scoped styles), and `module.js` (optional logic). This structure follows HubSpot's modular CMS pattern and allows sales ops to configure each module in the HubSpot editor without developer involvement.

Module	Responsibility
hero_banner	Full-bleed background image + logo overlay. CSS-only responsive scaling.
letterhead	Buyer details, sales rep email, quote number/date. Fetches <code>job_address</code> from API at runtime via deal ID.
included_benefits	Static checklist of pre-ticked benefits. Decorative only (disabled checkboxes).
line_items_table	Core interactive module. Renders two tables (Service / Upsell) filtered by description string. Manages checkboxes, descriptions, subtotals.
agreement	Terms checkbox + submit button. Click handler delegates to <code>QuoteAcceptanceManager</code> class.
footer	Configurable background image with position/size controls.

Client-Side State Management

The `line_items_table` module contains inline JavaScript that handles three primary concerns:

1. product description hydration (fetching `long_description` from the API for each line item by product ID),
2. checkbox state management (toggling Selected/Not Selected badges, adding/removing the faded class on upsell rows, tracking which products are selected), and
3. real-time totals (parsing currency values from DOM text nodes, summing selected amounts, calculating tax amount at 10% (adjustable variable), and formatting output using Intl.NumberFormat with the template's locale and currency).

QuoteAcceptanceManager

Defined in the base template's inline script, this is an ES6 class instantiated on `DOMContentLoaded`. Its lifecycle is as follows:

- `init()` - calls `checkQuoteStatus()`. If the quote is already accepted, calls `displayAcceptedQuote()`. Otherwise, calls `setupQuoteInteraction()`.

- `checkQuoteStatus()` - GET `/api/quotes/:no/status`. If accepted, fetches products via `/api/quotes/:no/products` and stores them.
- `displayAcceptedQuote()` - unchecks all checkboxes, disables them, then re-checks only previously selected products by matching on `product_sku` or `name`. Replaces agreement section with a read-only confirmation.
- `handleQuoteAcceptance()` - validates agreement checkbox, extracts product data from DOM rows (`extractProductDataFromRow`), calculates totals, POSTs to `/api/quotes/accept`, shows success overlay on 200.

Product data extraction uses three fallback strategies: direct currency element parsing, quantity-times-price pattern detection, and positional column indexing. This handles variations in HubSpot's line item rendering across locales and currencies.

Backend Architecture

Express Application Structure

The API follows a layered file structure:

- `config/` (environment variables, cache instance),
- `middleware/` (CORS, rate limiter, logger, error handler),
- `routes/` (deals, products, quotes, cache, health),
- `services/` (HubSpot API wrapper, invoice service), and
- `utils/` (calculation helpers, date utilities).

Middleware Pipeline

Requests pass through five middleware layers in order:

- **CORS:** In production, restricts to three HubSpot domain patterns derived from the account number (`hs-sites-ap1.com`, `hubspotpreview-ap1.com`, `hubspotpages-ap1.com`). In development, allows all origins.
- **Rate Limiter:** 1000 requests per 15-minute window per IP via `express-rate-limit`.
- **Body Parser:** `express.json()` + `express.urlencoded({ extended: true })`.
- **Logger:** Timestamps each request method and path.
- **Error Handler:** Catches unhandled errors, returns structured JSON with 500 status.

API Endpoints

Method	Path	Purpose
GET	/api/deals/:dealId	Fetch deal properties (job_address, stage). Cached.
GET	/api/products/:id	Full product data (sku, name, price, qty, amount). Cached.
GET	/api/products/:id/description	Name + description + long_description only. Cached.
POST	/api/products/batch	Fetch up to 50 products in parallel. Mixed cache/API.
GET	/api/quotes/:no/status	Search custom object for accepted quote by quote_no.
GET	/api/quotes/:no/products	Search selected products custom object by quote_no.
POST	/api/quotes/accept	Full acceptance flow (see below).
DELETE	/api/cache/clear	Flush all or specific cache key.
GET	/health	Status + cache stats.

Quote Acceptance Flow (POST /api/quotes/accept)

This is the most complex endpoint. It orchestrates multiple HubSpot API calls in a specific sequence:

1. **Duplicate check:** Searches Accepted Quotes custom object for existing record with the same `quote_no` where `is_quote_accepted = true`. Returns 409 if found.
2. **Create Accepted Quote:** POST to CRM with properties: `quote_no`, `is_quote_accepted`, `accepted_total_amount`, `subtotal_amount`, `gst_amount`, `accepted_at` (ISO timestamp), `deal_id`, `contact_id`, `company_id`, `ip_address`, `browser_info`.
3. **Generate invoices:** Calls `invoiceService.createInvoices()`. Creates two invoices (50% deposit due immediately, 50% balance due on deal `completion_date`). Each invoice gets its own line item, and associations to contact, company, deal. Status set to 'open' if contact exists.
4. **Create associations:** Parallel promises link the accepted quote to deal, contact, company, and original quote via labelled association endpoints (v3). Falls back to default associations (v4) on failure.
5. **Create Selected Products:** Iterates `selected_products` array sequentially. Each creates a custom object record with `quote_no`, `product_sku`, `name`, `description`, `quantity`, `price`, `amount`, `tax_amount`, and `discount` fields.

6. **Associate products to quote:** Parallel Promise.all linking each Selected Product to the Accepted Quote.
7. **Update original quote:** PATCH the original HubSpot quote's `hs_stats` to 'accepted'.

Response codes: 200 (full success), 207 (partial: some products failed), 409 (duplicate), 500 (no products created or system error).

HubSpot Service Layer

The hubspot.js service module wraps all CRM API calls through a central `makeRequest()` function that handles authentication (Bearer token), error classification (429 rate limit, 404 not found, 401 auth failure), and response parsing. It exposes focused functions: `getDeal()`, `getLineItem()`, `searchObject()`, `createObject()`, `updateObject()`, `createAssociation()`, `checkExistingQuote()`, `updateOriginalQuoteStatus()`, and `getDealCompletionDate()`.

The association logic deserves attention: it first attempts the v3 labeled association endpoint (PUT `/crm/v3/objects/{from}/{fromId}/associations/{to}/{toId}/{label}`), and on failure falls back to the v4 default association endpoint. This handles cases where custom association labels have not been defined in the HubSpot portal.

Invoice Service

The invoice service implements a specific business rule: every accepted quote generates exactly two invoices with a 50/50 split of the total (GST-inclusive) amount. The deposit invoice is due immediately. The balance invoice is due on the deal's `completion_date` property (fetched via a separate API call), falling back to immediately if no date is set.

Each invoice follows a four-step creation pattern:

1. create draft invoice (POST `/crm/v3/objects/invoices`),
2. create line item with the invoice amount and a tax rate group reference,
3. associate line item to invoice (typeId 409), associate contact (177), company (179), and deal (175), then
4. transition status from draft to open.

Caching Strategy

NodeCache is configured with environment-aware TTLs: 3600 seconds in production, 10 seconds in development. Cache keys are namespaced by entity type and ID (e.g., `product_12345`, `description_12345`, `deal_67890_prop1_prop2`). The batch products endpoint checks cache first for each ID, then fetches only uncached products from HubSpot in parallel, storing results before returning. A manual cache clear endpoint exists for operational use.

Data Model

Custom Objects

Object	Key Properties	Associations	Purpose
Accepted Quotes	quote_no, is_quote_accepted, accepted_total_amount, subtotal_amount, gst_amount, accepted_at, ip_address, browser_info	Deals, Contacts, Companies, Quotes, Selected Products	Immutable acceptance record with audit data
Selected Products	quote_no, product_sku, name, description, quantity, price, amount, tax_amount, discount_percentage, discount_amount	Accepted Quotes	Individual line-level record of what was selected

Association Graph

The accepted quote acts as the central node. It associates outward to:

- deal (accepted_quotes_to_deals),
- contact (accepted_quotes_to_contacts),
- company (accepted_quotes_to_companies),
- original quote (accepted_quotes_to_quotes), and
- each selected product (accepted_quotes_to_selected_products).

Invoices independently associate to deal, contact, company, and their own line items via HubSpot-defined type IDs.

Security Considerations

- **CORS:** Production whitelist restricted to three HubSpot hosting domain patterns for the specific portal. Credentials enabled in production only.
- **Rate limiting:** 1000 requests per 15 minutes per IP. Configurable via express-rate-limit.
- **Token isolation:** HubSpot access token stored in environment variables, never exposed to the client. All CRM calls proxied through the API.
- **Duplicate prevention:** Server-side check before creating any records. Returns 409 with the original acceptance timestamp.

- **Input validation:** Batch endpoint enforces array type check and 50-item maximum. Accept endpoint requires `quote_no` and non-empty `selected_products` array.
- **Audit capture:** IP address (from req.ip or client payload) and user-agent stored on each acceptance record.

Key Technical Decisions

Vanilla JS over a framework

The client-side code runs inside HubSpot's CMS rendering pipeline, which injects the page into an existing HTML document. Introducing React or Vue would add build tooling complexity, bundle size, and hydration issues for what is fundamentally a read-heavy page with limited interactivity. The `QuoteAcceptanceManager` class provides sufficient structure for the scope.

Custom API instead of HubSpot serverless functions

HubSpot serverless functions have a 6-second execution limit, restrictive middleware options, and no persistent caching. The quote acceptance flow involves 10+ sequential API calls that can exceed that limit. A standalone Express API provides full control over timeouts, caching, error handling, and middleware.

Sequential product creation instead of batch API

HubSpot's batch create API for custom objects has less granular error handling. By creating products sequentially, the system can return a 207 partial success with per-product error details, allowing the acceptance to proceed even if individual products fail.

In-memory cache instead of Redis

Single-instance deployment on Render makes Redis an unnecessary operational dependency. NodeCache provides sufficient TTL-based caching for product descriptions and deal data that rarely change during a session. Cache invalidation is available via the `/api/cache/clear` endpoint.

Conclusion

This system replaces a manual, multi-step quote-to-invoice pipeline with an automated, auditable flow triggered by a single customer action. The architecture keeps the frontend lightweight and CMS-native while delegating business logic and CRM orchestration to a dedicated API layer. The separation allows independent scaling, monitoring, and iteration of each concern.